



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e  
INTERACCIÓN HUMANO COMPUTADORA



## **REPORTE DE PRÁCTICA N° 09**

**NOMBRE COMPLETO:** Del Toro Ortiz Juan Pablo Yasbeth

**N° de Cuenta:** 317031814

**GRUPO DE LABORATORIO:** 13

**GRUPO DE TEORÍA:** 7

**SEMESTRE 2026-2**

**FECHA DE ENTREGA LÍMITE:** 23/04/2026

**CALIFICACIÓN:** \_\_\_\_\_

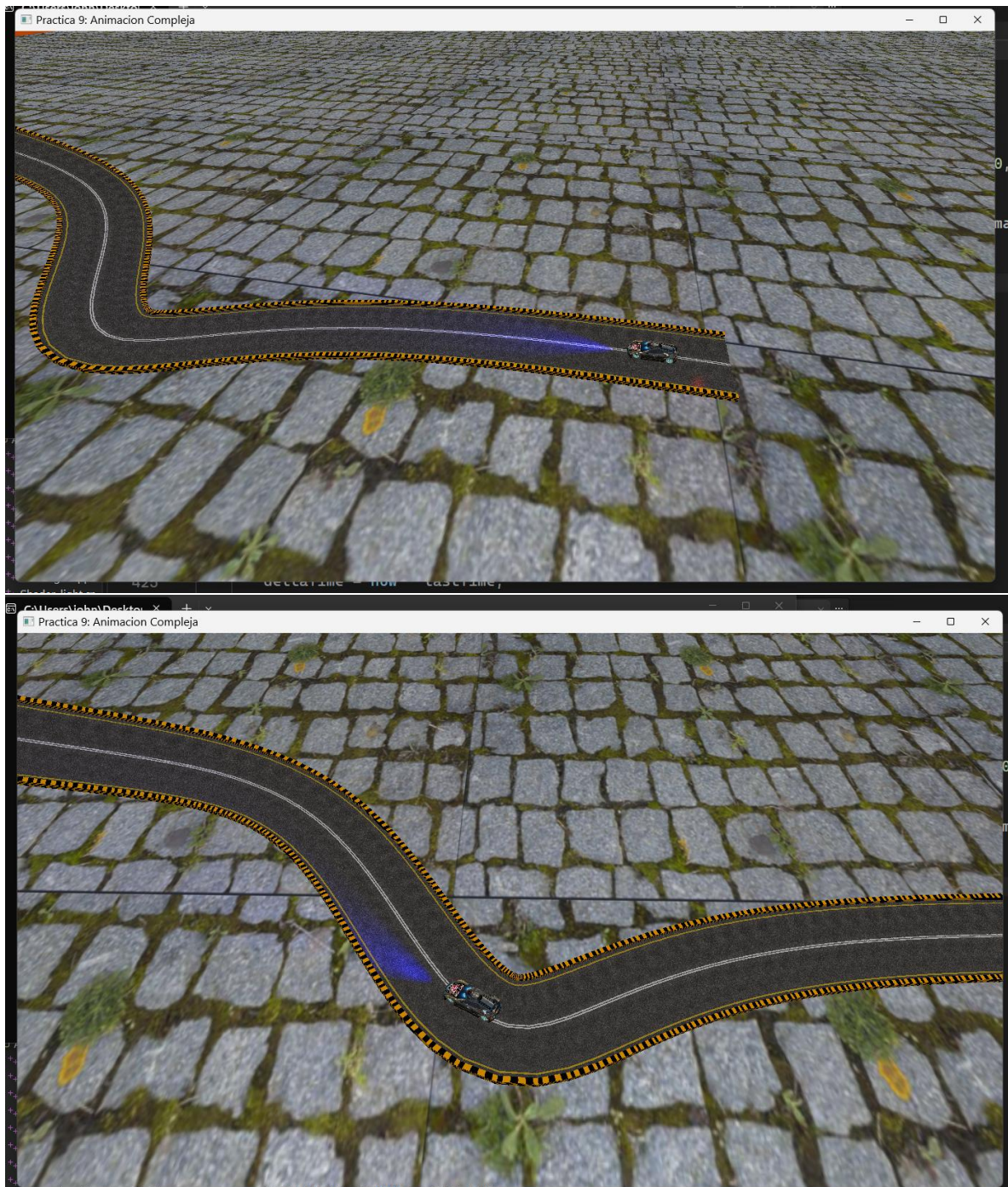
## REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

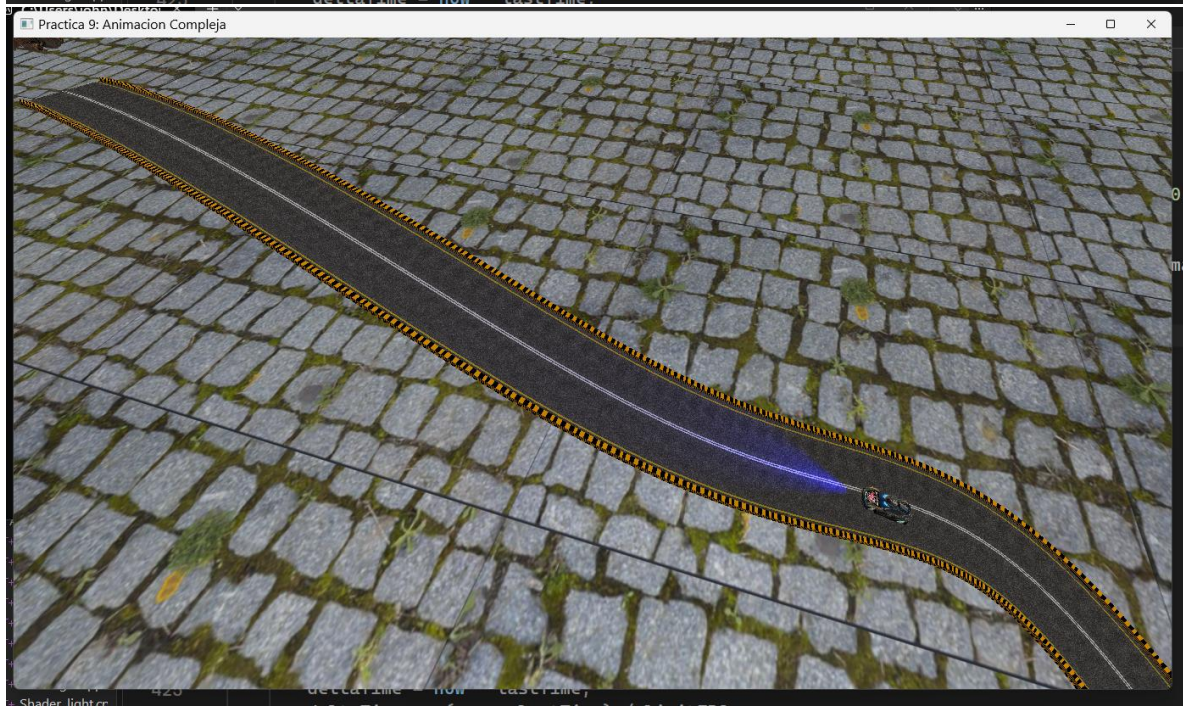
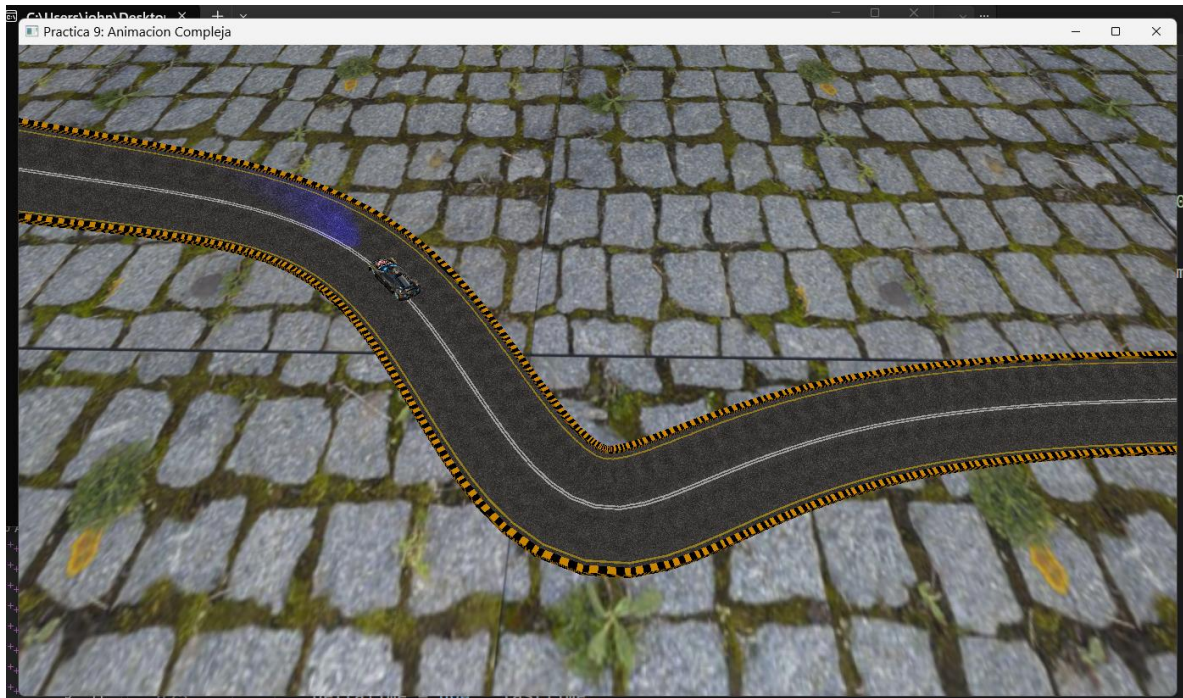
### Instrucciones:

- Ejercicio 1.- Hacer que su carro recorra la pista **Adecuadamente (curvas, inclinación en rampa, faro frontal ilumina la pista)** desde el inicio hasta el punto extremo y ahí se detenga. Se puede repetir la animación si se presiona una tecla designada.
- Ejercicio 2.-Separar de la Nave el otra ala y las dos hélices, Hacer que sobrevuele la pista **adecuadamente (jerarquía, aleteo, giro de hélices, faro ilumina la pista)** pero en sentido contrario, inicia en el punto extremo y aterriza junto al punto inicial. Esta animación no se puede repetir.
  - Extra 5 puntos: Su dado de 8 caras cae girando sobre el piso y muestra un número random **adecuadamente** hacia arriba, se repite la tirada al presionar una tecla.

## Capturas de pantalla Actividad 1:







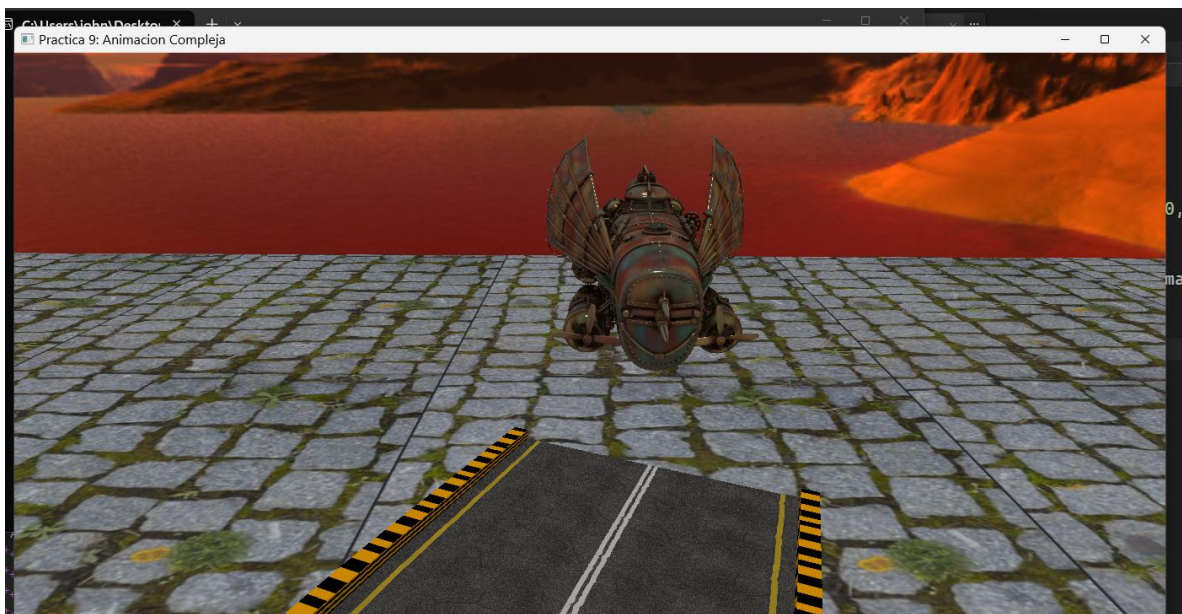




Video del funcionamiento:

[https://youtu.be/bjRsDL4\\_6KA](https://youtu.be/bjRsDL4_6KA)

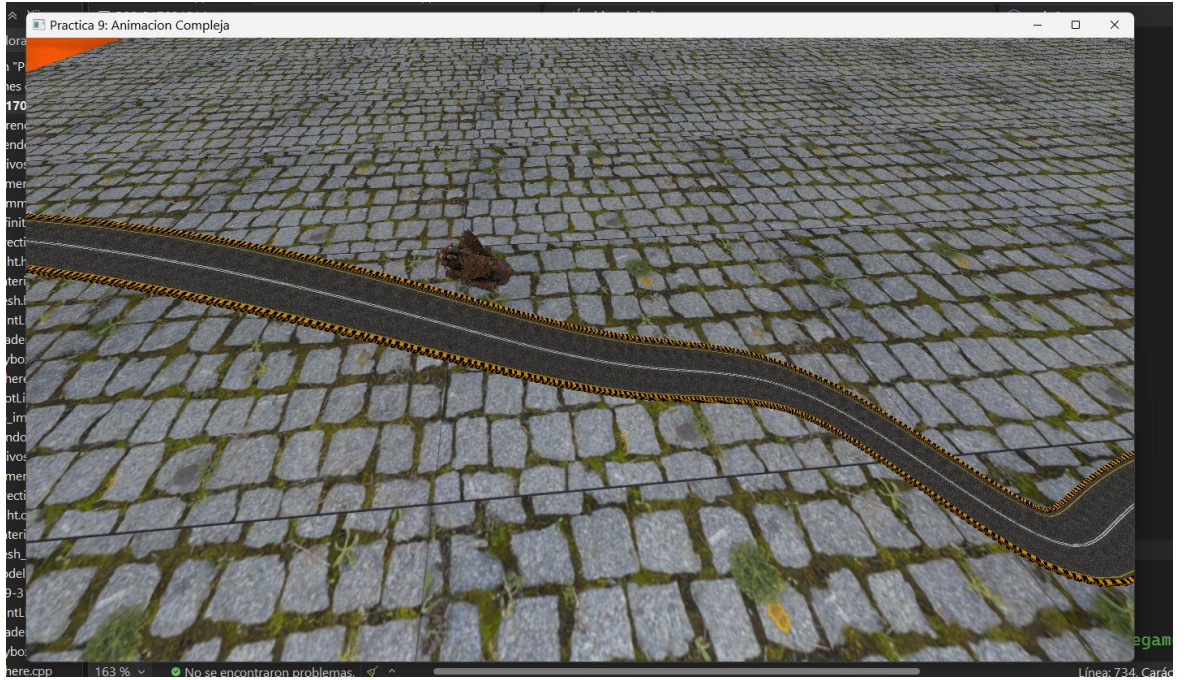
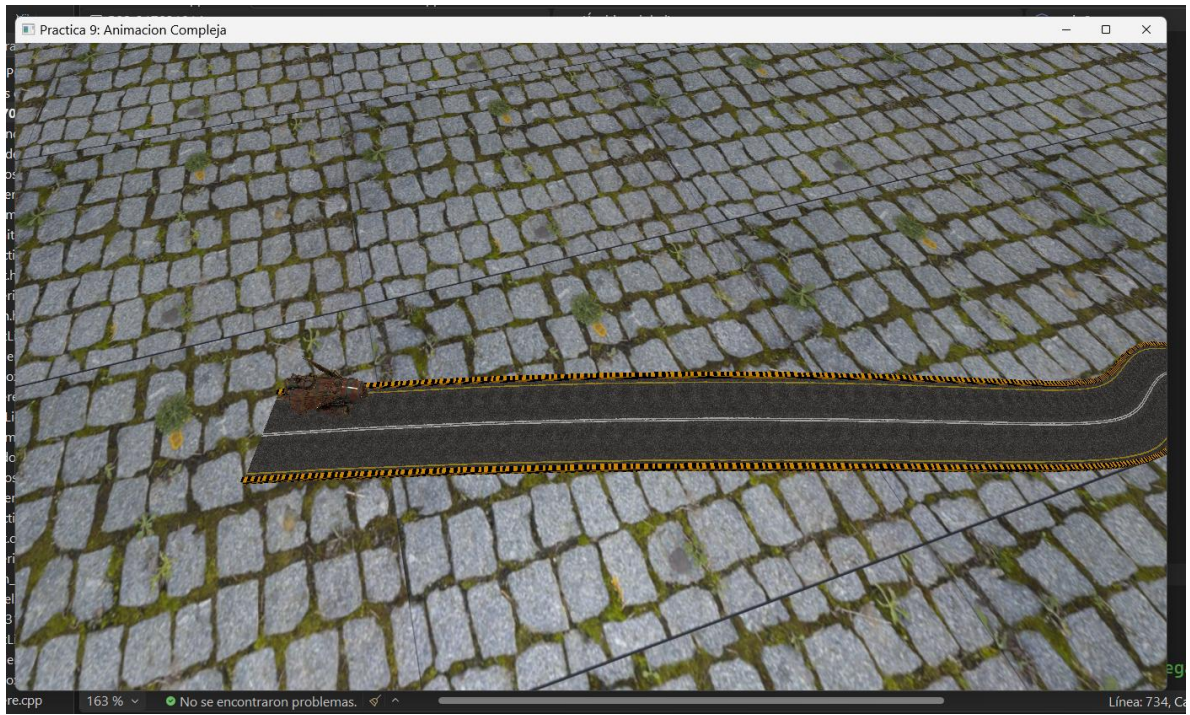
## Capturas de pantalla Actividad 2:















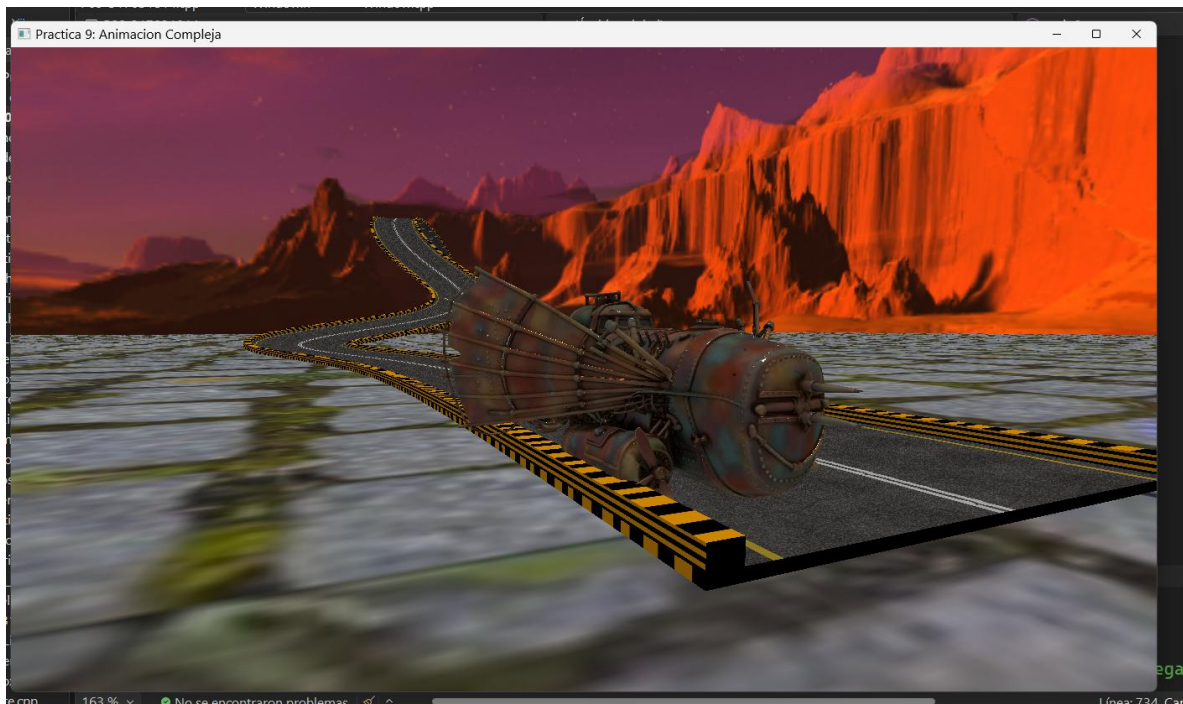


ctica 9: Animacion Compleja



ctica 9: Animacion Compleja





Video del funcionamiento:

<https://youtu.be/3Xksv4aDe3g>



## Código generado:

Código de modificaciones en Window.h

```
29 // Lógica de reinicio con tecla R
30 bool getReiniciarAnimacion() { return reiniciarAnimacion; }
31 void apagarReinicio() { reiniciarAnimacion = false; }
32
33 //Lógica presión con tecla T
34 bool getAccionT() { return accionT; }
35 void apagarAccionT() { accionT = false; }
36
```

```
66 bool reiniciarAnimacion;
67 bool accionT;
```

Código de modificaciones en Window.cpp

```
15
16 // Estado inicial
17 luzPrendida = true;
18 colorCiclo = 0;
19 reiniciarAnimacion = false; // Tecla R
20 accionT = false;           // Inicialización tecla T
21
```

```
125 // --- LÓGICA DE REINICIO (Tecla R) ---
126 if (key == GLFW_KEY_R && action == GLFW_PRESS)
127 {
128     theWindow->reiniciarAnimacion = true;
129 }
130
131 // --- LÓGICA DE ACCIÓN (Tecla T) ---
132 if (key == GLFW_KEY_T && action == GLFW_PRESS)
133 {
134     theWindow->accionT = true;
135 }
```

## Código de las variables

```
56 // --- VARIABLES AUTO ---
57 glm::vec3 posAuto = glm::vec3(0.0f, -1.8f, 3.5f); // Posición inicial
58 glm::vec3 dirAuto = glm::vec3(-1.0f, 0.0f, 0.0f); // Dirección inicial (Hacia -X)
59 float rotAutoY = 0.0f;
60 float rotAutoX = 0.0f;
61 float rotAutoZ = 0.0f;
62 float distanciaRecorrida = 0.0f;
63
64 // --- VARIABLES NAVE ---
65 glm::vec3 posNave = glm::vec3(-182.0f, 30.0f, -22.0f); // Posición inicial
66 glm::vec3 dirNave = glm::vec3(1.0f, 0.0f, 0.0f); // Dirección inicial (Hacia +X)
67 float rotNaveY = 0.0f;
68 float rotNaveX = 0.0f;
69 float rotNaveZ = 0.0f;
70 float distanciaNave = 0.0f;
71 bool naveIniciada = false;
```

## Código modelos:

```
88
89 // --- AUTO PRÁCTICA 9 ---
90 Model Cuerpo_M;
91 Model Cofre_M;
92 Model LlantaD1_M;
93 Model LlantaD2_M;
94 Model LlantaT1_M;
95 Model LlantaT2_M;
96
97
98 Model Pista_M;
99
100 // --- NAVE MODIFICADA ---
101 Model Nave_M;
102 Model AlaIzq_M;
103 Model AlaDer_M;
104 Model HeliceIzq_M;
105 Model HeliceDer_M;
```



```

316 // --- CARGA COCHE HELLDIVERS ---
317 Cuerpo_M.LoadModel("Models/cuerpo_auto_texturizado.obj");
318 Cofre_M.LoadModel("Models/cofre_auto_fi_logo.obj");
319 LlantaD1_M.LoadModel("Models/llantad1_texturizada.obj");
320 LlantaD2_M.LoadModel("Models/llantad2_texturizada.obj");
321 LlantaT1_M.LoadModel("Models/llantat1_texturizada.obj");
322 LlantaT2_M.LoadModel("Models/llantat2_texturizada.obj");
323
324 Pista_M = Model();
325 Pista_M.LoadModel("Models/pista.obj");
326
327
328 // --- CARGA DE NAVE Y SUS COMPONENTES ---
329 Nave_M = Model();
330 Nave_M.LoadModel("Models/nave_modificada.obj"); // El cuerpo base
331 AlaIzq_M = Model();
332 AlaIzq_M.LoadModel("Models/nave_ala_I.obj");
333 AlaDer_M = Model();
334 AlaDer_M.LoadModel("Models/nave_ala_D.obj");
335 HeliceIzq_M = Model();
336 HeliceIzq_M.LoadModel("Models/nave_helice_I.obj");
337 HeliceDer_M = Model();
338 HeliceDer_M.LoadModel("Models/nave_helice_D.obj");

```

Código de luces:

```

386 // Luz frontal del auto (Azul)
387 spotLights[2] = SpotLight(0.0f, 0.0f, 1.0f, // Color
388 2.0f, 2.0f, // Intensidad ambiente y difusa
389 0.0f, 0.0f, 0.0f, // Posición
390 0.0f, 0.0f, -1.0f, // Dirección
391 0.7f, 0.025f, 0.025f, // Coeficientes de atenuación
392 20.0f); // Ángulo de apertura (aprox 20°)
393 spotLightCount++;
394
395 // Luz de la Nave
396 spotLights[3] = SpotLight(1.0f, 0.0f, 0.0f, // Color
397 2.0f, 2.0f, // Intensidad ambiente y difusa
398 0.0f, 0.0f, 0.0f, // Posición
399 0.0f, 0.0f, -1.0f, // Dirección
400 0.7f, 0.025f, 0.025f, // Coeficientes de atenuación
401 20.0f); // Ángulo de apertura (aprox 20°)
402 spotLightCount++;

```

Código Lógica del reinicio del auto:

```

431 // --- LÓGICA DE REINICIO DE AUTO (Tecla R) ---
432 if (mainWindow.getReiniciarAnimacion()) {
433     // Reinicio Auto
434     posAuto = glm::vec3(0.0f, -1.8f, 3.5f);
435     rotAutoY = 0.0f;
436     rotAutoX = 0.0f;
437     rotAutoZ = 0.0f;
438     distanciaRecorrida = 0.0f;
439     rotllanta = 0.0f;
440
441     // Apagamos la bandera de R
442     mainWindow.apagarReinicio();
443 }

```

Código Lógica de movimiento del auto:

```
448 // Nueva máxima total: 195 unidades
449 if (distanciaRecorrida < 195.0f) {
450     distanciaRecorrida += velocidad;
451     rotllanta += rotllantaOffset * deltaTime;
452
453     // 1. Definición de Secciones de Animación
454     if (distanciaRecorrida <= 10.0f) { /* Recto inicial */ }
455     else if (distanciaRecorrida > 10.0f && distanciaRecorrida <= 35.0f) { /* Sec 1 */
456         float t_giro1 = (distanciaRecorrida - 10.0f) / 25.0f;
457         rotAutoY = t_giro1 * -18.0f;
458     }
459     else if (distanciaRecorrida > 35.0f && distanciaRecorrida <= 40.0f) { /* Sec 2 */ }
460     else if (distanciaRecorrida > 40.0f && distanciaRecorrida <= 50.0f) { /* Sec 3 */
461         float t_subida = (distanciaRecorrida - 40.0f) / 10.0f;
462         rotAutoY = -18.0f;
463         rotAutoZ = t_subida * -5.0f;
464     }
465     else if (distanciaRecorrida > 50.0f && distanciaRecorrida <= 65.0f) { /* Sec 4 */
466         float t_giro2 = (distanciaRecorrida - 50.0f) / 15.0f;
467         rotAutoY = -18.0f + (t_giro2 * 73.0f);
468         rotAutoZ = -5.0f + (t_giro2 * -5.0f);
469         rotAutoX = t_giro2 * -15.0f;
470     }
471     else if (distanciaRecorrida > 65.0f && distanciaRecorrida <= 75.0f) { /* Sec 5 */
472         rotAutoY = 55.0f;
473         rotAutoZ = -10.0f;
474         rotAutoX = -15.0f;
475     }
476     else if (distanciaRecorrida > 75.0f && distanciaRecorrida <= 90.0f) { /* Sec 6 */
477         float t_giro3 = (distanciaRecorrida - 75.0f) / 15.0f;
478         rotAutoY = 55.0f - (t_giro3 * 61.0f);
479         rotAutoZ = -10.0f + (t_giro3 * -8.0f);
480         rotAutoX = -15.0f + (t_giro3 * 21.0f);
481     }
482     else if (distanciaRecorrida > 90.0f && distanciaRecorrida <= 105.0f) { /* Sec 7 */
483         float t_sec6 = (distanciaRecorrida - 90.0f) / 15.0f;
484         rotAutoY = -6.0f + (t_sec6 * 1.0f);
485         rotAutoZ = -18.0f + (t_sec6 * -13.0f);
486         rotAutoX = 6.0f - (t_sec6 * 2.0f);
487     }
488     else if (distanciaRecorrida > 105.0f && distanciaRecorrida <= 115.0f) { /* Sec 8 */
489         float t_sec7 = (distanciaRecorrida - 105.0f) / 10.0f;
490         rotAutoY = -5.0f + (t_sec7 * 1.0f);
491         rotAutoZ = -31.0f;
492         rotAutoX = 4.0f;
493     }
494     else if (distanciaRecorrida > 115.0f && distanciaRecorrida <= 118.0f) { /* Sec 9 */
495         float t_sec8 = (distanciaRecorrida - 115.0f) / 3.0f;
496         rotAutoY = -4.0f + (t_sec8 * 1.0f);
497         rotAutoZ = -31.0f + (t_sec8 * 11.0f);
498         rotAutoX = 4.0f - (t_sec8 * 4.0f);
499     }
500     else if (distanciaRecorrida > 118.0f && distanciaRecorrida <= 123.0f) { /* Sec 10 */
501         float t_sec9 = (distanciaRecorrida - 118.0f) / 5.0f;
502         rotAutoY = -3.0f;
503         rotAutoZ = -20.0f - (t_sec9 * 3.0f);
504         rotAutoX = 0.0f;
505     }
```



```

506     else if (distanciaRecorrida > 123.0f && distanciaRecorrida <= 128.0f) { /* Sec 11 */
507         rotAutoY = -3.0f;
508         rotAutoZ = -22.0f;
509         rotAutoX = 0.0f;
510     }
511     else if (distanciaRecorrida > 128.0f && distanciaRecorrida <= 135.0f) { /* Sec 12 */
512         float t_sec12 = (distanciaRecorrida - 128.0f) / 7.0f;
513         rotAutoY = -3.0f + (t_sec12 * 3.0f);
514         rotAutoZ = -22.0f + (t_sec12 * 7.0f);
515         rotAutoX = 0.0f;
516     }
517     else if (distanciaRecorrida > 135.0f && distanciaRecorrida <= 145.0f) { /* Sec 13 */
518         float t_sec13 = (distanciaRecorrida - 135.0f) / 10.0f;
519         rotAutoY = 0.0f + (t_sec13 * 12.0f);
520         rotAutoZ = -15.0f + (t_sec13 * 13.0f);
521         rotAutoX = 0.0f;
522     }
523     else if (distanciaRecorrida > 145.0f && distanciaRecorrida <= 150.0f) { /* Sec 14 */
524         float t_sec14 = (distanciaRecorrida - 145.0f) / 5.0f;
525         rotAutoY = 12.0f + (t_sec14 * 5.0f);
526         rotAutoZ = -3.5f + (t_sec14 * 4.0f);
527         rotAutoX = 0.0f;
528     }
529     else if (distanciaRecorrida > 150.0f && distanciaRecorrida <= 155.0f) { /* Sec 15 */
530         float t_sec15 = (distanciaRecorrida - 150.0f) / 5.0f;
531         rotAutoY = 12.0f + (t_sec15 * 5.0f);
532         rotAutoZ = 2.0f + (t_sec15 * 1.4f);
533         rotAutoX = 0.0f;
534     }
535 }
536
537     else if (distanciaRecorrida > 160.0f && distanciaRecorrida <= 165.0f) { /* Sec 16 */
538         rotAutoY = 17.0f;
539         rotAutoZ = 0.0f;
540         rotAutoX = 0.0f;
541     }
542
543     // --- SECCIÓN 17: DESCENSO FINAL Y GIRO AJUSTADO (165 a 195) ---
544     else if (distanciaRecorrida > 165.0f && distanciaRecorrida <= 195.0f) {
545         rotAutoY = 19.0f;
546         rotAutoZ = 5.0f;
547         rotAutoX = 0.0f;
548     }
549
550     // 2. Cálculo de dirección (180 + 18 = 198 grados)
551     float anguloDireccion = (180.0f + rotAutoY) * toRadians;
552     dirAuto.x = cos(anguloDireccion);
553     dirAuto.z = sin(anguloDireccion);
554
555     // 3. Desplazar posición física
556     if (distanciaRecorrida > 40.0f) {
557         if (distanciaRecorrida > 165.0f) {
558             // Descenso con 5 grados de inclinación
559             posAuto.y -= (float)(sin(abs(rotAutoZ) * toRadians) * velocidad);
560         }
561         else {
562             // Subida original
563             posAuto.y += (float)(sin(abs(rotAutoZ) * toRadians) * velocidad);
564         }
565     }
566 }

```

```

548 // 2. Cálculo de dirección (180 + 18 = 198 grados)
549 float anguloDireccion = (180.0f + rotAutoY) * toRadians;
550 dirAuto.x = cos(anguloDireccion);
551 dirAuto.z = sin(anguloDireccion);
552
553 // 3. Desplazar posición física
554 if (distanciaRecorrida > 40.0f) {
555     if (distanciaRecorrida > 165.0f) {
556         // Descenso con 5 grados de inclinación
557         posAuto.y -= (float)(sin(abs(rotAutoZ) * toRadians) * velocidad);
558     }
559     else {
560         // Subida original
561         posAuto.y += (float)(sin(abs(rotAutoZ) * toRadians) * velocidad);
562     }
563 }
564 posAuto.x += dirAuto.x * velocidad;
565 posAuto.z += dirAuto.z * velocidad;
566
567 }
568 else {
569     // --- ESTADO FINAL ESTÁTICO (Unidad 195) ---
570     rotAutoY = 19.0f;
571     rotAutoZ = 5.0f;
572     rotAutoX = 0.0f;
573 }
574

```

Código Lógica del inicio movimiento de la Nave:

```

575 // --- LÓGICA DE INICIO ÚNICO DE NAVE (Tecla T) ---
576 // Añadimos "!naveIniciada" para que solo entre si la nave NO ha comenzado
577 if (mainWindow.getAccionT() && !naveIniciada) {
578     // Posición inicial (aseguramos que esté en el origen de la animación)
579     posNave = glm::vec3(-182.0f, 30.0f, -22.0f);
580     rotNaveY = 0.0f;
581     rotNaveX = 0.0f;
582     rotNaveZ = 0.0f;
583     distanciaNave = 0.0f;
584
585     // Activamos el movimiento
586     naveIniciada = true;
587
588     // Apagamos la bandera de la ventana
589     mainWindow.apagarAccionT();
590 }
591 else if (mainWindow.getAccionT() && naveIniciada) {
592     // Opcional: Si se presiona T y ya está iniciada, solo se apaga la bandera
593     // de la ventana para que no se quede acumulada, pero no hace nada.
594     mainWindow.apagarAccionT();
595 }

```



Códigos de variables responsables del movimiento de alas y hélices:

```
597 // Variables para la rotación de hélices y alas
598 float rotHeliceContinua = glfwGetTime() * 800.0f;
599 float anguloAleteo = 20.0f * sin(glfwGetTime() * 10.0f);
600
```

Código de Lógica de la nave:

```
601 // --- LOGICA MOVIMIENTO DE LA NAVE (Sección 10: Nivelación rápida en 10 unidades) ---
602 float velNave = movOffset * deltaTime;
603 // Límite total: 190.0
604 if (naveIniciada && distanciaNave < 190.0f) {
605     distanciaNave += velNave;
606     if (distanciaNave <= 55.0f) { /* Sec 1 */
607         rotNaveY = 19.0f;
608         rotNaveZ = 0.0f;
609     }
610     else if (distanciaNave > 55.0f && distanciaNave <= 85.0f) { /* Sec 2 */
611         float t_sec2 = (distanciaNave - 55.0f) / 30.0f;
612         rotNaveY = 19.0f + (t_sec2 * -35.0f);
613         rotNaveZ = t_sec2 * -30.0f;
614     }
615     else if (distanciaNave > 85.0f && distanciaNave <= 95.0f) { /* Sec 3 */
616         rotNaveY = -16.0f;
617         rotNaveZ = -30.0f;
618     }
619     else if (distanciaNave > 95.0f && distanciaNave <= 110.0f) { /* Sec 4 */
620         float t_sec4 = (distanciaNave - 95.0f) / 15.0f;
621         rotNaveY = -16.0f + (t_sec4 * 35.0f);
622         rotNaveZ = -30.0f;
623     }
624     else if (distanciaNave > 110.0f && distanciaNave <= 122.0f) { /* Sec 5 */
625         float t_sec5 = (distanciaNave - 110.0f) / 12.0f;
626         rotNaveY = 19.0f + (t_sec5 * 27.0f);
627         rotNaveZ = -30.0f + (t_sec5 * 5.0f);

```

```

629     else if (distanciaNave > 122.0f && distanciaNave <= 130.0f) { /* Sec 6 */
630         float t_sec6 = (distanciaNave - 122.0f) / 8.0f;
631         rotNaveY = 46.0f;
632         rotNaveZ = -25.0f + (t_sec6 * 25.0f);
633     }
634     else if (distanciaNave > 130.0f && distanciaNave <= 143.0f) { /* Sec 7 */
635         float t_sec7 = (distanciaNave - 130.0f) / 13.0f;
636         rotNaveY = 46.0f + (t_sec7 * -61.0f);
637         rotNaveZ = 0.0f;
638     }
639     else if (distanciaNave > 143.0f && distanciaNave <= 160.0f) { /* Sec 8 */
640         float t_sec8 = (distanciaNave - 143.0f) / 17.0f;
641         rotNaveY = -15.0f + (t_sec8 * 15.5f);
642         rotNaveZ = t_sec8 * -5.0f;
643     }
644     else if (distanciaNave > 160.0f && distanciaNave <= 180.0f) { /* Sec 9 */
645         float t_sec9 = (distanciaNave - 160.0f) / 20.0f;
646         rotNaveY = 0.5f + (t_sec9 * -5.5f);
647         rotNaveZ = -5.0f;
648     }
649     // SECCIÓN 10: (180 a 190) - NIVELACIÓN Z (10 unidades)
650     else {
651         float t_sec10 = (distanciaNave - 180.0f) / 10.0f;
652         rotNaveY = -5.0f;
653         // Z: De -5.0 a 0.0 en un tramo más corto
654         rotNaveZ = -5.0f + (t_sec10 * 5.0f);
655     }
656     rotNaveX = 0.0f;
657     // --- FÍSICA ---
658     float anguloDireccionNave = rotNaveY * toRadians;
659     dirNave.x = cos(anguloDireccionNave);
660     dirNave.z = sin(anguloDireccionNave);
661     posNave.x += dirNave.x * velNave;
662     posNave.z += dirNave.z * velNave;
663     if (rotNaveZ != 0.0f) {
664         posNave.y += (float)(sin(rotNaveZ * toRadians) * velNave);
665     }
666 }
667 else if (naveIniciada && distanciaNave >= 190.0f) {
668     // ESTADO FINAL FIJO
669     rotNaveY = -5.0f;
670     rotNaveZ = 0.0f;
671 }

```



## Código instancia del coche

```
735 // --- INSTANCIA DEL COCHE (HELLDIVERS) ---
736 model = glm::mat4(1.0);
737 model = glm::translate(model, posAuto);
738 model = glm::rotate(model, -rotAutoY * toRadians, glm::vec3(0.0f, 1.0f, 0.0f)); // Agregamos el signo '-'
739 model = glm::rotate(model, rotAutoX * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
740 model = glm::rotate(model, rotAutoZ * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
741
742 // Rotación de corrección base (mantenla como está si ya estaba alineado)
743 model = glm::rotate(model, -90.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
744 modelaux = model;
745
746 // --- ACTUALIZACIÓN DE LUZ FRONTAL (SPOTLIGHT 2) ---
747 glm::vec4 offsetFrente = glm::vec4(0.0f, 0.5f, 2.0f, 1.0f);
748 glm::vec3 posLuzAuto = glm::vec3(modelaux * offsetFrente);
749 glm::vec3 dirLuzReal = glm::vec3(modelaux * glm::vec4(0.0f, 0.0f, 1.0f, 0.0f));
750 spotLights[2].SetFlash(posLuzAuto, glm::normalize(dirLuzReal));
751 shaderList[0].SetSpotLights(spotLights, spotLightCount);
752
753 // --- RENDERIZADO DEL CUERPO ---
754 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
755 Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
756 color = glm::vec3(1.0f, 1.0f, 1.0f);
757 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
758 Cuerpo_M.RenderModel();
759
760 // --- COFRE ---
761 model = glm::translate(modelaux, glm::vec3(0.0f, 0.72f, 1.1f));
762 model = glm::rotate(model, 0 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
763 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
764 Cofre_M.RenderModel();
765
766 // --- LLANTAS CON JERARQUÍA ---
767 float wheelOffsets[4][3] = {
768     {-0.5f, 0.3f, 1.4f}, // Delantera Izq
769     {0.5f, 0.3f, 1.4f}, // Delantera Der
770     {-0.5f, 0.3f, -1.5f}, // Trasera Izq
771     {0.5f, 0.3f, -1.5f} // Trasera Der
772 };
773
774 for (int i = 0; i < 4; i++) {
775     model = glm::translate(modelaux, glm::vec3(wheelOffsets[i][0], wheelOffsets[i][1], wheelOffsets[i][2]));
776     // Las llantas rotan sobre su propio eje X local
777     model = glm::rotate(model, rotllanta * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
778     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
779
780     if (i == 0) LlantaD1_M.RenderModel();
781     else if (i == 1) LlantaD2_M.RenderModel();
782     else if (i == 2) LlantaT1_M.RenderModel();
783     else LlantaT2_M.RenderModel();
784 }
785
```

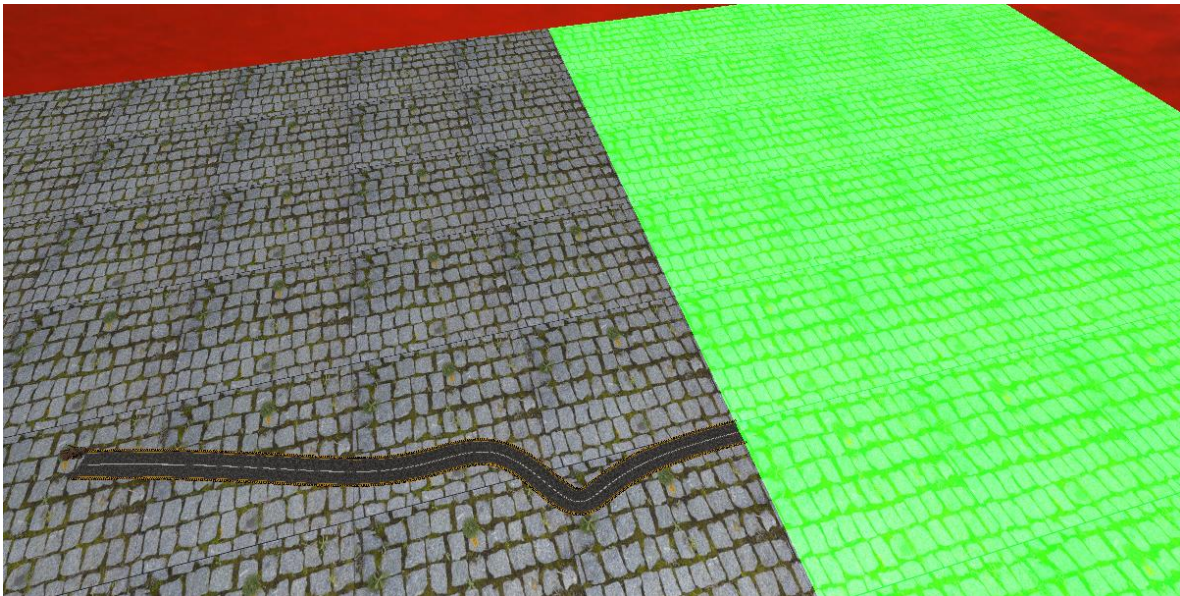
## Código Intancia de la Nave

```
787 // --- RENDERIZADO CUERPO NAVE ---
788 model = glm::mat4(1.0);
789 model = glm::translate(model, posNave);
790 model = glm::rotate(model, -rotNaveY * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
791 model = glm::rotate(model, rotNaveX * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
792 model = glm::rotate(model, rotNaveZ * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
793
794 // Corrección base para alinear el modelo original (el 69.0f que ya usabas)
795 model = glm::rotate(model, 90.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
796 model = glm::scale(model, glm::vec3(3.5f, 3.5f, 3.5f));
797 modelaux = model; // Matriz base para las alas y hélices jerárquicas
798
799 //Hay un error extraño que hace que al poner la luz esta cargue mal y openGL estalle en colores
800 /*
801 // ===== LUZ DE LA NAVE =====
802 glm::vec4 offsetLuzNave = glm::vec4(0.0f, -1.2f, 0.0f, 1.0f);
803 glm::vec3 posLuzNave = glm::vec3(modelaux * offsetLuzNave);
804 glm::vec3 dirLuzNave = glm::vec3(0.0f, -1.0f, 0.0f);
805 |
806 spotLights[3].SetFlash(posLuzNave, dirLuzNave);
807 shaderList[0].SetSpotLights(spotLights, spotLightCount);
808 */
809 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
810 Nave_M.RenderModel();
811
812 // --- ALA DERECHA (JERARQUÍA RELATIVA CON ALETEO) ---
813 model = modelaux;
814 model = glm::translate(model, glm::vec3(-0.2f, 0.0f, 0.3f));
815 model = glm::rotate(model, anguloAleteo * toRadians, glm::vec3(0.0f, 1.0f, 1.0f));
816 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
817 AlaDer_M.RenderModel();
818
819 // --- ALA IZQUIERDA (JERARQUÍA RELATIVA CON ALETEO) ---
820 model = modelaux;
821 model = glm::translate(model, glm::vec3(0.2f, 0.0f, 0.3f));
822 model = glm::rotate(model, -anguloAleteo * toRadians, glm::vec3(0.0f, 1.0f, 1.0f));
823 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
824 AlaIzq_M.RenderModel();
825
826 // --- HÉLICE DERECHA ---
827 model = modelaux;
828 model = glm::translate(model, glm::vec3(-0.35f, -0.3f, 0.37f));
829 model = glm::rotate(model, rotHeliceContinua * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
830 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
831 HeliceDer_M.RenderModel();
832
833 // --- HÉLICE IZQUIERDA ---
834 model = modelaux;
835 model = glm::translate(model, glm::vec3(0.35f, -0.3f, 0.37f));
836 model = glm::rotate(model, -rotHeliceContinua * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
837 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
838 HeliceIzq_M.RenderModel();
```

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

El principal error al que me afronte fue en la actividad 2 ya que si bien ya tenía mi modelo y le había podido poner una luz como se solicitaba al momento de mandarla a llamar y hacer el movimiento por algún motivo estallaba la luz cubriendo la mitad del escenario, honestamente intente emplear intenté usar IA Generativa (Google Gemini) para corregir el error de destello, pero por cuestiones de tiempo no me fue posible solucionar el error y simplemente comente el código.

Otras problemáticas fueron dificultad para hacer rotaciones en z con el auto ya que a veces el modelo traspasaba la pista y además tuve que descargar el modelo de la nave de la fuente original ya que no podía editar el modelo proporcionado por el profesor y esto a su vez conllevó a que al exportar el modelo tuviera problemas con su sistema de coordenadas, pero ambos problemas se solucionaron con ensayo y error.



Error 1: las texturas no cargan correctamente la transparencia



---

## Propmts empleados para la corrección problemas:

---

**Yo:** Hola, Necesito ayuda ya que no logro visualizar mi luz que está ligada al objeto nave de mi código y cuando “carga” la luz esta explota y hace ver la mitad del escenario verde, por favor para ayudarme básate en: <https://learnopengl.com/Lighting/Basic-Lighting>

### Google Gemini:



Entiendo perfectamente. El problema de que el escenario se vea verde o la luz "explote" suele ocurrir por un error en el cálculo de la atenuación o por enviar valores de color fuera del rango  $[0, 1]$  al shader, lo que satura los buffers de color.

Basándome en **LearnOpenGL** y en la estructura de tu proyecto, aquí tienes la sección de código corregida para posicionar y enviar la luz ligada a tu nave. He ajustado los parámetros de **atenuación** (Constant, Linear, Quadratic) para que la luz sea visible pero controlada, evitando ese efecto de "explosión".

Copia esta sección en tu ciclo de renderizado donde manejas las luces:

C++



```
// --- LÓGICA DE LUZ LIGADA A LA NAVE (Basado en Basic Lighting - LearnOpenGL) --
// Se posiciona la luz ligeramente al frente de la nave para que ilumine el camino
glm::vec3 offsetLuzNave = glm::vec3(dirNave.x * 2.0f, 0.5f, dirNave.z * 2.0f);
glm::vec3 posicionLuzNave = posNave + offsetLuzNave;

// Definimos un color blanco con intensidad controlada para evitar saturación (el
glm::vec3 colorLuzNave = glm::vec3(1.0f, 1.0f, 1.0f);

// Enviamos los parámetros a la PointLight dedicada a la nave
// Los valores 0.3f, 0.5f, 0.01f controlan que la luz no sea infinita ni explote
pointLights[2].SetColor(colorLuzNave.r, colorLuzNave.g, colorLuzNave.b);
pointLights[2].SetPos(posicionLuzNave.x, posicionLuzNave.y, posicionLuzNave.z);
pointLights[2].SetAttenuation(1.0f, 0.05f, 0.005f); // Atenuación suave para gran
pointLights[2].SetIntensity(0.8f, 1.0f); // Intensidad ambiental y difusa equilib
```

// Nota: Asegúrate de que el shader esté recibiendo los vectores de posición  
// en View Space o World Space según tu implementación de Basic Lighting.

### ¿Por qué fallaba antes?

1. **Atenuación:** Si los valores `linear` o `quadratic` son 0, la luz no pierde fuerza con la distancia y satura el escenario.
2. **Posicionamiento:** Al usar el vector `dirNave`, la luz ahora siempre apunta hacia donde se dirige el modelo, simulando faros reales en lugar de una explosión estática de color.

### Nota:

La conversación con Gemini siguió, pero no logre solucionar el problema a si que solo comente el código.

### 3.- Conclusión:

- a. Los ejercicios del reporte: Complejidad, Explicación.

En esta ocasión la práctica si me costó mucho trabajo ya que por algún motivo la luz de la nave provocaba errores extraños en la escena, pero no pude solucionarlos, y respecto a los modelos y sus sistemas de orígenes nuevamente estos me causaron problemáticas al intentar trabajar en el espacio.

- b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica

En referencia a detalles de la explicación teórica me pareció un poco escueta, aunque comprendo que solo es el tema introductorio de animación, no obstante, hubiera querido un poco más de explicación teórica.

- c. Conclusión de la práctica:

De esta práctica puedo concluir que el manejo de animación requiere de contener mas de una transformación y no solo una rotación o traslación, por o cual para esta práctica usamos elementos móviles como ruedas, hélices o alas para poder cumplir con dicho requicito

Otro elemento que quisiera destacar es como el manejo de condicionales permite simular el manejo de animaciones en conjunto con lo anteriormente mencionado, ya que al unir traslaciones,

rotaciones y elementos móviles a la par podemos simular el proceso de animación sin llegar a serlo.

## Bibliografía

- *LearnOpenGL - Basic Lighting*. (n.d.). <https://learnopengl.com/Lighting/Basic-Lighting>